

PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Profundización — Lambdas y Streams en Java

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

Ocho líneas de código para comparar dos números. Una interfaz funcional y una lambda lo resuelven en una sola.



TRANSICIÓN

Cerrando la unidad con un vistazo hacia adelante

En la sesión de `Comparable` viste que una interfaz con un solo método puede describir un comportamiento (cómo comparar dos objetos). Java aprovecha justamente esas interfaces de un solo método para permitir una sintaxis mucho más corta: las **expresiones lambda**. Hoy, como cierre de la Unidad 2, das un vistazo a esta forma más moderna y concisa de escribir código orientado a objetos, y a la API de **Streams** que se apoya en ella.

CONCEPTO

Interfaz funcional: el requisito para usar lambdas

Una **interfaz funcional** es una interfaz con **exactamente un** método abstracto (puede tener constantes, pero solo un método por implementar). `Comparable` que ya conoces es una; Java también trae varias más listas para usar, como `Comparator<T>` , `Runnable` y `Predicate<T>` . La anotación `@FunctionalInterface` es opcional, pero documenta la intención.

```
@FunctionalInterface
public interface Operacion {
    int calcular(int a, int b);
}
```

CONCEPTO

La expresión lambda: una implementación sin nombre

Una **lambda** es una forma corta de implementar el único método de una interfaz funcional, sin necesitar una clase completa ni el nombre del método. La sintaxis es (parámetros) -> cuerpo .

```
Operacion suma = (a, b) -> a + b;  
Operacion resta = (a, b) -> a - b;  
  
System.out.println(suma.calcular(4, 3)); // 7  
System.out.println(resta.calcular(4, 3)); // 1
```

Java infiere que `suma` y `resta` son objetos que implementan `Operacion` , sin necesidad de escribir `new Operacion() { ... }` .

CONTRA EJEMPLO

Lo que una lambda no puede reemplazar

Una lambda solo funciona con una interfaz que tenga **un único** método abstracto. Si la interfaz tiene dos o más métodos por implementar, Java no puede saber cuál de ellos representa el cuerpo de la lambda — en ese caso, sigue siendo obligatorio implementarla con una clase completa.

```
public interface Volador {  
    void volar();  
    void aterrizar(); // segundo método: ya no es interfaz funcional  
}  
// Volador v = () -> ...; // Error: no se puede usar lambda aquí
```



CONCEPTO

Comparator con lambda: ordenar sin implementar una clase

En vez de implementar `Comparable` dentro de la clase (como hiciste con `Estudiante`), `Comparator<T>` permite definir un criterio de orden **por fuera**, como una lambda, y pasarlo directamente a `sort()`.

```
ArrayList<Estudiante> estudiantes = new ArrayList<>();  
// ... agregar estudiantes ...  
  
estudiantes.sort((e1, e2) -> Double.compare(e1.getPromedio(), e2.getPromedio()));
```

CONCEPTO

La API de Streams: procesar colecciones en cadena

Un **Stream** es una secuencia de datos (originada normalmente en una colección) sobre la que se encadenan operaciones, sin modificar la colección original. Se obtiene con `.stream()` y las operaciones intermedias reciben, casi siempre, una lambda.

```
ArrayList<String> nombres = new ArrayList<>(List.of("Ana", "Beto", "Camila", "Dan"));

nombres.stream()
    .filter(n -> n.length() > 3)
    .forEach(System.out::println);
```


CONCEPTO

Operaciones intermedias: `filter`, `map`, `sorted`

Las operaciones intermedias reciben un stream y devuelven **otro stream**, permitiendo encadenarlas — no ejecutan nada hasta que llega una operación terminal.

Operación	Qué hace
<code>filter(predicado)</code>	Conserva solo los elementos que cumplen una condición
<code>map(función)</code>	Transforma cada elemento en otro valor
<code>sorted()</code>	Ordena los elementos del stream



CONCEPTO

Operaciones terminales: `forEach`, `collect`, `count`

Una operación terminal es la que finalmente **ejecuta** todo lo encadenado antes y produce un resultado — sin ella, las operaciones intermedias nunca se disparan.

```
List<String> nombresLargos = nombres.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList()); // "collect" arma la lista final

long total = nombres.stream().filter(n -> n.length() > 3).count();
```



CASO

Todo junto: promedios por encima del corte

Sobre la lista de `Estudiante` del sistema de biblioteca/notas que has usado en la unidad, un reporte típico se resuelve con una sola cadena de stream: filtrar los que aprueban, ordenarlos de mayor a menor promedio, y recolectar sus nombres.

```
List<String> aprobados = estudiantes.stream()
    .filter(e -> e.getPromedio() >= 3.0)
    .sorted((e1, e2) -> Double.compare(e2.getPromedio(), e1.getPromedio()))
    .map(Estudiante::getNombre)
    .collect(Collectors.toList());
```

TENSIÓN CONCEPTUAL

¿Más corto siempre es mejor?

Una cadena de Streams con cinco o seis operaciones encadenadas puede ser difícil de leer y de depurar comparada con el mismo código escrito con un `for` tradicional. La ganancia real de Streams está en la **claridad de intención** (filtrar, transformar, contar) cuando la cadena es corta — no en comprimir todo el programa en una sola línea a costa de que nadie más la entienda.

INTERACCIÓN

Escriban su primera cadena de Streams

En parejas: partiendo de una `ArrayList<Pagable>` (Factura y Nomina de sesiones anteriores), escriban una cadena de **stream** que filtre los elementos con `calcularMonto() > 100000`, los ordene de mayor a menor monto, y los recolecte en una nueva lista.

6 minutos · comparen su solución con la de otra pareja

SÍNTESIS

Lo que hay que llevarse de hoy — y de la Unidad 2

1. Una interfaz funcional tiene exactamente un método abstracto, y eso permite implementarla con una lambda en vez de una clase completa.
2. Un Stream encadena operaciones intermedias (`filter` , `map` , `sorted`) que no se ejecutan hasta que llega una operación terminal (`forEach` , `collect` , `count`).
3. Lambdas y Streams son una forma más concisa de expresar lo mismo que ya sabes hacer con clases, interfaces y polimorfismo — no un paradigma aparte.
4. Con esto cierra la Unidad 2: de clases y objetos a herencia, interfaces, polimorfismo, entrada/salida, diagramas UML y esta profundización funcional.



CIERRE

Antes de la próxima unidad

- Reescribir con Streams un ejercicio propio de la unidad que antes hayan resuelto con un `for` y un `if` anidado.
- Repasar los diagramas de clases hechos en la sesión pasada — se retoman como base de diseño en la siguiente unidad.

Con esta sesión se cierra la Unidad 2 — Paradigma Objetual.

